

MotifVM: An Invariant-Checked Reasoning Runtime

Core Freeze at the Adapter Boundary and ExtractedFact Normal Form

Nikit Phadke

Version 0.5.4

Abstract

LLM applications frequently collapse ingestion, reasoning, execution, and trust into a single opaque loop. A model reads an artifact, emits text, calls tools, and the surrounding application often treats the output as actionable state. MotifVM proposes a different architecture: a small cognitive virtual machine in which artifacts are first normalized into hash-bound evidence references and extracted facts, reasoning components propose typed state patches, and the runtime alone validates, authorizes, applies, verifies, commits, and exports terminal states. This paper describes the MotifVM v0.5.4 core freeze. The main contribution is not a new prompt pattern or workflow graph, but a kernel boundary: **ArtifactAdapter** $\rightarrow$ **EvidenceRef** $\rightarrow$ **ExtractedFact** $\rightarrow$ **StatePatch** $\rightarrow$ invariants $\rightarrow$ terminal state $\rightarrow$ audit pack. We present the runtime model, adapter contract, fact normal form, patch authorization system, invariant-driven terminal semantics, audit-pack verifier, implemented domains, evaluation gates, threat model, and limitations.

1 Introduction

Modern LLM systems are commonly described as agents, chains, or workflows. These terms emphasize orchestration: what component calls what, in what order, with what prompt. They are less precise about the semantics of state. When a model summarizes a spreadsheet, reviews a code diff, or interprets a document, where does the evidence live? Which component is allowed to mutate application state? How is a failed answer represented? Can the result be independently checked after export?

MotifVM starts from the opposite direction. It treats reasoning as a virtual-machine problem. The runtime has explicit memory, typed patches, authority references, invariants, terminal states, and portable audit packs. LLMs and tools are outside the trust boundary. They may propose outputs, but they do not directly write truth into the runtime.

The design target is an auditable cognitive VM:

Model and tool outputs do not become truth. They become proposed state transitions over explicit cognitive state.

The v0.5.4 milestone freezes the core boundary around artifact adapters and an **ExtractedFact** normal form. This is the point at which MotifVM’s kernel becomes separable from future surface expansion. CSV, diff, repository, PDF, log, API, and email adapters should all behave like device drivers: they translate the outside world into stable evidence and facts without changing the kernel.

2 Design Goals

MotifVM is designed around eight goals:

1. **Explicit state.** Reasoning state must be inspectable as a structured object, not implicit in chat history.
2. **Single mutation path.** All state changes must pass through `StatePatch`.
3. **Adapter isolation.** Artifact ingestion must be separated from domain reasoning.
4. **Evidence stability.** Every evidence record must be hash-bound and locator-stable.
5. **Fact/claim separation.** Extracted facts are weaker than claims and must not imply terminal conclusions.
6. **Patch authorization.** LLMs, passes, tools, reconciliation logic, and runtime code must have different permissions.
7. **Invariant terminal semantics.** Success and failure must be committed terminal states with typed failure classes.
8. **Portable verification.** Audit packs must be checkable without rerunning the original task.

3 System Overview

The core MotifVM transition law is:

$$\text{CognitiveState} + \text{StatePatch} \rightarrow \text{validate} \rightarrow \text{authorize} \rightarrow \text{apply} \rightarrow \text{verify} \rightarrow \text{commit}$$

The artifact ingestion law is:

$$\text{Artifact} \rightarrow \text{ArtifactAdapter} \rightarrow \text{EvidenceRef}^* \rightarrow \text{ExtractedFact}^*$$

Domain passes then consume extracted facts and propose claims through patches:

$$\text{ExtractedFact}^* \rightarrow \text{DomainPass} \rightarrow \text{StatePatch} \rightarrow \text{Claim}^*$$

The complete path is therefore:

$$\begin{aligned} &\text{ArtifactAdapter} \rightarrow \text{EvidenceRef} \rightarrow \text{ExtractedFact} \rightarrow \text{StatePatch} \\ &\rightarrow \text{Invariant} \rightarrow \text{TerminalState} \rightarrow \text{AuditPack} \rightarrow \text{PackVerifier} \end{aligned}$$

4 Kernel Primitives

Primitive	Role
<code>CognitiveState</code>	Explicit runtime memory containing task AST, graph, artifacts, invariants, pass history, patch timeline, and terminal metadata.
<code>StatePatch</code>	The only mutation vehicle for cognitive state.
<code>PatchAuthorization</code>	Role-based permission boundary for patches.
<code>ArtifactAdapter</code>	Device-driver-like ingestion component for external artifacts.
<code>EvidenceRef</code>	Hash-bound pointer to a stable region of an input artifact.
<code>ExtractedFact</code>	Normalized fact extracted from evidence before domain claims are made.
<code>Claim</code>	Runtime assertion produced by a domain pass and traceable to evidence.
<code>Invariant</code>	Predicate over state that determines correctness, warnings, or terminal failure.
<code>AuthorityRef</code>	Source-bound rule reference with section metadata and hashes.
<code>TerminalState</code>	Committed success or structured committed failure.
<code>AuditPack</code>	Portable bundle containing state, graph, facts, lineage, invariants, inputs, and reports.
<code>PackVerifier</code>	Independent consistency checker for exported audit packs.
<code>PatchTimeline</code>	Ordered record of authorized and rejected patch proposals.

5 CognitiveState

`CognitiveState` is the VM memory image. In v0.5.4 it contains:

- task AST and domain metadata,
- authority references,
- input manifest,
- motif vector state,
- reasoning graph,
- artifacts,
- decisions,
- invariant results,
- pass history,
- patch timeline,
- execution log,
- terminal status, reason, and failure class.

The state object is deliberately verbose. The goal is not minimal storage; the goal is inspectability. A committed state should explain not only what answer was produced, but which inputs, evidence, facts, authorities, invariants, and patches produced it.

6 Artifact Adapters

An artifact adapter is the boundary between unstructured or semi-structured external artifacts and the VM. It is analogous to a device driver in an operating system. A disk driver does not change the kernel's process model; it gives the kernel a disciplined way to interact with a disk. Similarly, a CSV adapter or PDF adapter should not change MotifVM's reasoning semantics; it should produce stable evidence and normalized facts.

6.1 Adapter Contract

Each adapter has:

- an adapter identifier,
- a version,
- supported media or path types,
- a deterministic `adapt(input_ref, path)` function.

Each adapter emits an `AdapterResult`:

- adapter id and version,
- input id,
- path,
- content hash,
- zero or more resolved input artifacts,
- `EvidenceRef[]` records,
- `ExtractedFact[]` records,
- adapter metadata.

Adapters may not mutate `CognitiveState`. They may not emit terminal claims. They may not mark invariants as passed. They are perception components, not reasoning components.

6.2 Built-In Adapters

v0.5.4 implements three adapters:

Adapter	Input	Facts emitted
<code>adapter:csv:v0.5</code>	CSV files	<code>dccb_component</code> facts
<code>adapter:git_diff:v0.5</code>	Patch files	<code>code_added_line</code> facts
<code>adapter:repo:v0.5</code>	Repository directories	<code>code_added_line</code> , <code>repo_changed_file</code> facts

7 EvidenceRef

`EvidenceRef` is the stable evidence pointer. Its purpose is to preserve a durable relation between an extracted item and the artifact region it came from.

```
EvidenceRef {  
  id  
  inputId  
  inputHash  
  locatorType  
  locator  
  excerpt  
  metadata  
}
```

Examples of locator types include rows, lines, cells, byte ranges, JSON paths, and file regions. The input hash matters because a line number without a content hash is not evidence; it is only a coordinate.

8 ExtractedFact Normal Form

`ExtractedFact` is the normal form between raw evidence and domain reasoning.

```
ExtractedFact {  
  id  
  kind  
  value  
  evidenceRefId  
  confidence  
  extractedBy  
  domainHints  
  metadata  
}
```

Facts are not claims. A fact says what was extracted from an artifact. A claim says what a domain pass concludes from facts under authority and invariants. This distinction prevents domain assumptions from leaking into adapters.

8.1 Examples

Artifact fragment	ExtractedFact	Possible claim
tier1,120	dccb_component	Computed CRAR uses Tier I capital.
return True	code_added_line	Authorization bypass detected.
auth.py changed	repo_changed_file	Reviewed file is hash-bound.

9 Facts to Claims

Domain passes consume extracted facts and propose claims through `StatePatch`. In the DCCB domain, component facts become CRAR computation claims. In the code-review domain, added-line facts become security findings and risk claims.

This stage is where domain knowledge enters. The adapter does not know that a CRAR threshold matters. The DCCB pass does. The adapter does not know that `return True` is unsafe in an authorization function. The code-review pass does. The runtime then checks whether the proposed claims satisfy invariants.

10 StatePatch Semantics

`StatePatch` is the syscall-like mutation interface for MotifVM. A patch may contain:

- nodes to add,
- nodes to update,
- edges to add,
- artifacts to add,
- decisions to add,
- task updates,
- status update,
- motif support deltas.

Before a patch is applied, the runtime checks:

1. schema validity,
2. graph endpoint validity,
3. duplicate node constraints,
4. motif delta validity,
5. role authorization.

11 Patch Authorization

Patch authorization is a kernel-level concern because every actor must obey it. v0.5.4 uses role policies:

Role	Allowed	Blocked examples
Domain pass	Add domain-scoped claims, outputs, edges, and artifacts.	Cross-domain claim injection.
LLM	Add bounded narrative artifacts.	Terminal status, invariant results, input manifest, authority refs, direct reconciliation patches.
Runtime	Commit state and write runtime metadata.	Unverified terminal facts.
Reconciliation	Add non-mutating correction proposals.	Source mutation without confirmation.

The important point is that LLM output can remain useful without being trusted as state. A model may summarize the final result, but it may not silently change the terminal status or mark failed invariants as passed.

12 Reasoning Graph

The reasoning graph connects evidence, claims, outputs, assumptions, decisions, and errors. Graph edges encode relations such as **supports**, **produces**, **contradicts**, and **blocks**. This graph is not decorative; invariants inspect it. Audit packs export it in JSON, DOT, and Mermaid forms.

The graph lets MotifVM answer questions such as:

- Which evidence supports this claim?
- Which authority references govern this invariant?
- Which claim produced this output?
- Which contradiction caused reconciliation?
- Which patch introduced this artifact?

13 Invariants and Terminal States

Invariants define whether a state can be committed as success or must be committed as structured failure. MotifVM distinguishes failure classes rather than collapsing all failures into generic exceptions.

Failure class	Meaning
<code>input_invalid</code>	Malformed or invalid artifact values.
<code>computation_blocked</code>	Required evidence is missing or insufficient.
<code>reconciliation_required</code>	Evidence conflicts and a non-mutating correction is proposed.
<code>security_risk_detected</code>	Code-review invariants detect unsafe behavior.
<code>llm_patch_rejected</code>	A model proposed a forbidden transition.
<code>authority_missing</code>	Required authority references are absent.
<code>lineage_missing</code>	Terminal outputs lack traceable evidence.

This makes failure part of the VM output. A failed run still preserves evidence, graph state, invariant failures, reconciliation artifacts, and audit metadata.

14 Authority References

`AuthorityRef` records source-bound rule references. v0.4.5 introduced section-level hardening:

- source hash,
- section id,
- quoted rule excerpt,
- effective date,
- retrieval timestamp,
- section hash.

This is intentionally snapshot-based. MotifVM does not browse live regulatory sources in the current kernel; it verifies checked-in authority snapshots.

15 Audit Packs

An audit pack is a portable evidence bundle for a committed state. v0.5.4 audit packs include:

- `state.json`,
- `report.md`,
- `graph.json`, `graph.dot`, `graph.mmd`,
- `lineage.json`,
- `invariants.json`,
- `inputs_manifest.json`,
- `extracted_facts.json`,
- `llm_calls.json`,
- `reconciliation_patch.json`,
- `patch_timeline.json`,
- `patch_timeline.md`,
- artifact JSON files.

16 Pack Verification

The pack verifier checks internal consistency without rerunning the task. It verifies:

- required files,
- input file and directory hashes,
- authority source hashes,
- authority section hashes,
- graph endpoints,
- lineage references,
- reconciliation policy,
- report terminal status consistency,
- patch timeline shape,
- adapter output conformance.

This changes the trust model from “trust my answer” to “inspect this evidence bundle and verify its internal consistency.”

17 Implementation

The current implementation is a dependency-light Python package. The runtime exposes commands for running tasks, rendering reports, exporting audit packs, comparing runs, running evaluations, running adversarial suites, verifying packs, and checking adapter conformance.

```
make test
make adapter-conformance
make demo
make eval
make adversarial
make adversarial-100
python3 -m motifvm verify-pack <audit-pack-dir>
```

The implementation includes two domain profiles:

- **DCCB CRAR audit:** computes CRAR, checks threshold compliance, detects reported versus computed conflicts, and proposes non-mutating reconciliation patches.
- **Code review:** reviews patch and repository inputs for security-sensitive findings such as authorization bypass, secret literals, shell execution, eval/exec, disabled TLS verification, SQL interpolation, and unsafe deserialization.

18 Evaluation

MotifVM’s release gates are intentionally modest but concrete. The goal is to verify runtime boundaries, not to claim universal domain performance.

Version	Core proof	Gate
0.1.8	Kernel and two-domain audit packs	Canonical demo runs
0.2.5	Provider boundary and evaluation	7-case eval harness
0.3.5	Adversarial reliability	20-case adversarial suite
0.4.5	Scale and integration	Repository review + adversarial-100
0.5.4	Adapter boundary freeze	Adapter conformance + extracted facts

The v0.5.4 release gate observed:

- 18 unit tests passing,
- adapter conformance passing for CSV, git diff, and repository adapters,
- demo suite passing,
- evaluation suite passing,
- 20-case adversarial suite passing,
- 100-case adversarial suite passing,
- representative audit pack verification passing,
- PDF generation passing.

19 Threat Model

MotifVM protects the following assets:

- cognitive state,
- input manifests and hashes,
- authority references and hashes,
- extracted facts and evidence refs,
- invariant results,
- terminal state,
- audit packs.

Threats considered include malformed input, adversarial numeric fields, unsafe code diffs, LLM attempts to modify terminal facts, unauthorized source mutation, missing lineage, and audit-pack tampering.

Controls include schema validation, patch authorization, input and authority hashing, domain and universal invariants, failure taxonomy, patch timeline export, and independent pack verification.

20 Limitations

MotifVM does not prove that every domain rule is complete, every source authority is legally current, or every possible security flaw is detected. It does not replace static analyzers, regulatory counsel, formal methods, or human review. It currently uses checked-in authority snapshots rather than live source retrieval. Repository review is still patch-oriented rather than a full program analysis engine.

The verifier checks internal consistency of the exported audit pack. It does not certify that the domain is complete or that the output is legally or financially sufficient.

21 Future Work

With the adapter boundary frozen, future work should primarily expand surfaces rather than mutate the kernel:

- XLSX, PDF, JSON, log, API, database, and email adapters,
- richer domain profiles,
- larger labeled evaluations,
- stronger static-analysis integration for code review,
- source snapshotting for external authorities,
- interactive audit-pack inspection UI,
- deployment patterns for CI and regulated workflows.

22 Conclusion

MotifVM is an invariant-checked reasoning runtime. Its core contribution is the disciplined treatment of model and tool output as proposed state transitions rather than truth. Artifacts enter through adapters. Adapters emit evidence and facts. Domain passes propose claims. StatePatches are validated and authorized. Invariants define terminal states. Audit packs preserve the trace. The verifier checks the exported bundle.

This architecture separates perception, reasoning, trust, mutation, and portability. That separation is the kernel. The rest is adapters, domains, invariants, tools, and interfaces.